

GitHub Copilot Instructions for Orion Content Repository

Repository Context

This is the Orion Content application - a Next.js-based content management system with Prisma ORM, PostgreSQL database, and Edge Runtime compatibility requirements.

Code Quality Standards

TypeScript Requirements

- Use strict TypeScript with proper type definitions
- Prefer interfaces over types for object shapes
- Use generic types for reusable components
- Always include return types for functions
- Use `const assertions` for immutable data structures

Next.js Best Practices

- Use App Router patterns (app/ directory structure)
- Implement proper Server Components vs Client Components separation
- Use `use client` directive only when necessary
- Prefer Server Actions for form handling
- Implement proper error boundaries and loading states

Database & Prisma Guidelines

- Always validate Prisma schema changes for compatibility
- Use proper transaction handling for complex operations
- Implement proper error handling for database operations
- Use Prisma's type-safe query methods
- Validate all database migrations before suggesting

Edge Runtime Compatibility

- Ensure all suggested code is Edge Runtime compatible
- Avoid Node.js-specific APIs in Edge functions
- Use Web APIs instead of Node.js APIs where possible
- Test compatibility with Vercel Edge Runtime constraints
- Validate third-party dependencies for Edge compatibility

Validation Rules

Dependency Management

- Always check package.json for existing dependencies before suggesting new ones
- Verify version compatibility with existing packages
- Prefer peer dependencies over direct dependencies when appropriate

- Check for security vulnerabilities in suggested packages
- Validate bundle size impact of new dependencies

Environment Compatibility

- Ensure code works across development, staging, and production environments
- Validate environment variable usage and fallbacks
- Check for proper configuration management
- Ensure secrets are properly handled and not exposed

Performance Requirements

- Optimize for Core Web Vitals (LCP, FID, CLS)
- Implement proper caching strategies
- Use dynamic imports for code splitting
- Optimize images and assets
- Minimize bundle size and runtime overhead

Code Review Checklist

Security Validation

- ☐ No hardcoded secrets or API keys
- ☐ Proper input validation and sanitization
- ☐ CSRF protection implemented where needed
- ☐ SQL injection prevention (Prisma handles this)
- ☐ XSS prevention in user-generated content

Performance Validation

- ☐ No unnecessary re-renders in React components
- ☐ Proper memoization where beneficial
- ☐ Efficient database queries (no N+1 problems)
- ☐ Proper error handling and loading states
- ☐ Bundle size impact assessed

Accessibility Requirements

- ☐ Proper ARIA labels and roles
- ☐ Keyboard navigation support
- ☐ Screen reader compatibility
- ☐ Color contrast compliance
- ☐ Focus management in dynamic content

Automated Validation Integration

Pre-commit Hooks

- TypeScript compilation check
- ESLint and Prettier formatting
- Prisma schema validation
- Dependency audit

- Test suite execution

CI/CD Pipeline Integration

- Automated dependency vulnerability scanning
- Environment compatibility testing
- Edge Runtime compatibility validation
- Performance regression testing
- Database migration validation

Error Handling Patterns

Client-Side Error Handling

```
// Use proper error boundaries
import { ErrorBoundary } from 'react-error-boundary'

// Implement proper loading states
const [loading, setLoading] = useState(false)
const [error, setError] = useState<string | null>(null)
```

Server-Side Error Handling

```
// Use proper try-catch with specific error types
try {
  const result = await prisma.user.findUnique({ where: { id } })
  return { success: true, data: result }
} catch (error) {
  if (error instanceof PrismaClientKnownRequestError) {
    return { success: false, error: 'Database error' }
  }
  throw error
}
```

Documentation Requirements

- Include JSDoc comments for all public functions
- Document complex business logic
- Provide usage examples for reusable components
- Maintain up-to-date README files
- Document API endpoints and their contracts

Testing Guidelines

- Write unit tests for utility functions
- Implement integration tests for API routes
- Use React Testing Library for component tests
- Mock external dependencies properly
- Maintain test coverage above 80%

Deployment Considerations

- Ensure all code is production-ready
- Validate environment-specific configurations
- Check for proper error logging and monitoring
- Verify database migration compatibility
- Test Edge Runtime deployment compatibility

Remember: Always prioritize code quality, security, and performance. When in doubt, prefer explicit, readable code over clever optimizations.